

BDS BOOTCAMP WEEK

Session 2: Git & GitHub

Version Control · Collaboration Workflow

How today works

- So far: Terminal, Python, and VS Code installed — we ran out of time before notebooks and the hands-on part
- First up today: finishing that — then straight into how to track changes and collaborate on your work
- This is the skill you'll use for every single assignment submission in this program
- It's a lot of new vocabulary at once — you don't need to have it all memorized by the end of today
- By the end: you'll have created your own repo, committed a change, and pushed it to GitHub

TODAY'S AGENDA

- Catch-up: notebooks
- Catch-up: hands-on & troubleshooting
- Why version control
- Installing Git
- Git basics
- Git vs. GitHub
- Markdown, README, .gitignore
- Python quick peek
- Hands-on: create your own repo & push

CATCH-UP

Working with Notebooks

Terminal script vs. notebook

- A .py script runs top to bottom, all at once — good for finished, reusable code
- A notebook (.ipynb) runs in cells you execute one at a time — good for exploring data step by step
- Most of your BDS assignments and data exploration will happen in notebooks
- You'll still write .py scripts later for anything meant to run as a pipeline or tool

RULE OF THUMB

Exploring / analyzing
→ notebook

Building something reusable
→ script

Jupyter basics: cells & kernel

- **Cell** A block you run independently — code or Markdown (text)
- **Run (Shift+Enter)** Executes the current cell and moves to the next
- **Kernel** The running Python process behind your notebook — holds all your variables
- **Restart kernel** Wipes all variables — a clean slate when things get messy
- **Cell order matters** Variables exist based on what you've run, not what's on screen

Google Colab vs. local Jupyter

Same notebook format (.ipynb) — different place it runs.

Google Colab

- Runs in your browser — nothing to install
- Free GPU access for heavier tasks later in the program
- Files live in your Google Drive by default
- Needs an internet connection

Local Jupyter (via VS Code)

- Uses your own "bds_venv" environment and packages
- Works offline, full control over your files
- Better for larger/private datasets
- This is what we'll mostly use in class

Saving & exporting

- Local Jupyter (via VS Code): Ctrl/Cmd+S saves the notebook, same as any other file
- Colab: autosaves to your Google Drive as you go — you don't really have to think about it
- When you're ready to hand something in: File → Download as → .pdf or .html gives a read-only copy

Good habits (that save you pain later)

- Run cells top to bottom — don't jump around and run things out of order
- Before trusting your results (or submitting), do: Restart Kernel → Run All
- Use Markdown cells to label sections — future you will thank present you
- Keep variable and function names short but meaningful (df, not x7)

Troubleshooting cheat sheet

"command not found" / "'python' is not recognized"

Restart your terminal after install, or ask a TA to check your PATH

Install stuck on Windows

Try running the installer as Administrator

Activation script won't run (Windows)

PowerShell blocks it by default — run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once, or just use cmd.exe

VS Code doesn't see your environment

In VS Code: Ctrl/Cmd+Shift+P → "Python: Select Interpreter" → choose the bds_venv one

PART 1

Why Version Control

You've lived this problem already

```
report_final.docx → report_final_v2.docx → report_final_v2_REALLY_FINAL.docx
```

- Git = a tool that tracks every change to your files over time, with full history
- You can always go back to an earlier version — nothing is ever truly lost
- Multiple people can work on the same project without overwriting each other
- It's an industry-standard skill — expected in every data/software job

PART 2

Git Basics

Installing Git

- Windows: download the installer from git-scm.com — default options are fine, just click through
- Mac: often already there — run `git --version` in a terminal; if it prompts to install Xcode Command Line Tools, accept that
- Already using Anaconda? `conda install git` works too if you'd rather not leave that ecosystem
- Verify: `git --version` in a terminal should print a version number

One-time setup: tell Git who you are

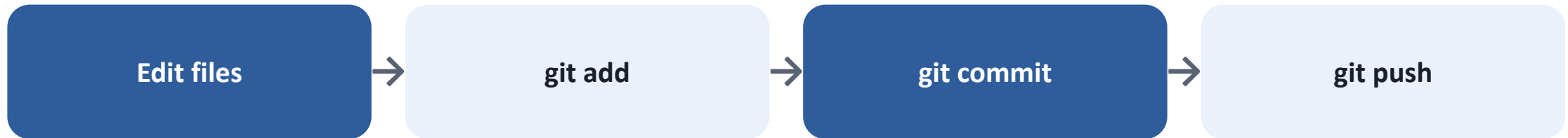
```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

The core commands

```
git init                # start tracking this folder
git status              # what changed?
git add file.py         # stage a file
git commit -m "message" # save a snapshot
git log                 # see the history
```

- "Staging" (git add) lets you choose exactly what goes into the next snapshot
- Write commit messages that explain WHY, e.g. "fix date parsing bug", not "stuff"

The daily loop



```
git pull          git clone <repo-url>      # copy a repo to your laptop
                  # get the latest changes
```

- Push = send your commits to GitHub · Pull = get everyone else's commits

PART 3

GitHub

Git vs. GitHub — not the same thing

Git

- The version control tool itself
- Runs locally on your laptop
- Works completely offline
- Installed as a program (comes with Anaconda / installed separately)

GitHub

- A website that hosts git repositories in the cloud
- Where you back up, share, and collaborate on your repos
- Adds extra tools: pull requests, issues, project pages
- This is where you'll find course materials & submit work

Creating your own repository

- On GitHub: click "New" → name your repo → choose Public or Private → Create (leave "Add a README" unchecked — start empty)
- Locally: git init connects a folder to git, and git remote add points it at your GitHub repo

```
git init                                # start tracking this folder
git add workshop_notebook.ipynb        # stage the notebook
git commit -m "first commit"            # save a snapshot
git remote add origin https://github.com/your-username/your-repo.git
git push -u origin main                 # send it to GitHub
```

README & .gitignore

- README.md — the front page of your repo, written in Markdown: what the project is, how to run it
- .gitignore — tells git which files to never track
 - Common ones to ignore: `__pycache__/, .ipynb_checkpoints/`, large data files, `.env` (secrets)
 - GitHub can generate a starter Python `.gitignore` for you when creating the repo

PART 4

Quick Python Primer

A tiny bit of Python (so today's file makes sense)

We haven't run a full Python session yet — just enough here to read the file you'll commit today.

```
name = "Alex" # a variable
print(f"Hello, {name}!") # prints text

for n in [1, 2, 3]: # runs the line below 3 times
    print(n)
```

- That's genuinely it for today — a proper Python pass comes once real coursework kicks in
- That exact code is in `workshop_notebook.ipynb` — the file you'll open, edit, and push in the hands-on

Your turn (with TA support)

- Make sure you have a GitHub account (create one now if not: github.com)
- Create a new repo on GitHub — name it, leave it empty (no README)
- Locally: `git init` in a new folder, save `workshop_notebook.ipynb` into it (download from Moodle), edit the name variable
- `git add + git commit`, then `git remote add origin <your-repo-url>` and `git push -u origin main`
- Check GitHub in your browser — confirm your commit shows up in the repo's history

First time pushing to GitHub may ask you to authenticate — a TA can help set this up.

COMMON ISSUES

Troubleshooting cheat sheet

"fatal: not a git repository"

You're not inside a folder that's been git init'd or cloned — check with `pwd/cd`

Push asks for a password and fails

GitHub needs a Personal Access Token or SSH key, not your account password — ask a TA

Forgot to stage a file before commit

Run `git add <file>` first, then `git commit` again

"error: remote origin already exists"

You already linked a remote — check it with `git remote -v`, or fix with `git remote set-url origin <url>`

WRAP-UP

By the end of today you should have...

- A GitHub account, set up to push from your laptop
- Created your own repo, committed a change, and pushed it
- Comfortable with add / commit / push without looking it up
- A clear sense of git vs. GitHub, and why version control matters

NEXT UP

Session 3 — Mon 7 Sept

AI tools, Hugging Face,
Obsidian & LaTeX

Thanks — see you Monday!

Questions? Ask now, or catch a TA on your way out.